

CNN-LSTM Model

Layer & Parameter Reference

Automatic Modulation Classification · PyTorch · RadioML 2016.10a / 2018.01a

Input (B, 2, L)	->	Conv Block 1 (B, 64, L/2)	->	Conv Block 2 (B, 128, L/4)	->	Transpose (B, L/4, 128)	->	BiLSTM (B, L/4, 256)	->	Mean Pool (B, 256)	->	Output (B, 24)
-----------------	----	---------------------------	----	----------------------------	----	-------------------------	----	----------------------	----	--------------------	----	----------------

This document is a layer-by-layer reference for the **CNNLSTM** class defined in **cnn_lstm.py** (PyTorch). Every module, every constructor argument, and every line of the forward pass is explained in plain language. This model differs from the Keras CNN1D guide in two important ways: it uses **PyTorch's nn.Module API** rather than Keras, and it adds a **bidirectional LSTM** after the convolutional feature extractor to capture long-range sequential dependencies in the I/Q signal.

Contents

1. Class signature & constructor arguments
 2. nn.Sequential conv block — overview
 3. Conv Block 1 — Conv1d, BatchNorm1d, ReLU, MaxPool1d, Dropout
 4. Conv Block 2 — deeper filters (conv_filters * 2)
 5. nn.LSTM — sequential modelling
 6. forward() — the data flow step by step
 7. Classifier head — LayerNorm, Dropout, Linear
 8. Key differences: PyTorch vs Keras
 9. Full model code
-

1. Class signature & constructor arguments

`__init__` — all hyperparameters are set here, nothing is hardcoded

```
class CNNLSTM(nn.Module):
    def __init__(
        self,
        num_classes: int = 24,
        input_channels: int = 2,
        conv_filters: int = 64,
        kernel_size: int = 5,
        lstm_hidden: int = 128,
        lstm_layers: int = 1,
        bidirectional: bool = True,
        dropout: float = 0.4,
    ) -> None:
        super().__init__()
```

nn.Module is the base class for all PyTorch neural networks. Inheriting from it gives the class parameter tracking, device management (`.to('cuda')`), state saving (`state_dict()`), and training/eval mode switching (`model.train()` / `model.eval()`). `super().__init__()` must always be called first to initialise these internals.

All architecture choices are exposed as constructor arguments with sensible defaults. This means you can instantiate different variants without modifying the class — e.g. a shallower unidirectional model with fewer filters for a quick experiment.

Constructor parameters

num_classes	24	Number of output neurons — one per modulation class. 24 covers RadioML 2018.01a. Use 11 for 2016.10a.
--------------------	----	---

input_channels	2	Number of input channels. Always 2 for I/Q signals (in-phase and quadrature). Would be 1 for real-valued input.
conv_filters	64	Number of filters in the first Conv1d layer. The second Conv1d block uses $\text{conv_filters} * 2 = 128$. This single value controls the width of the entire convolutional feature extractor.
kernel_size	5	Width of every convolutional kernel in timesteps. Larger than the Keras model (which uses 3) — captures slightly wider local patterns at the cost of more parameters. $\text{padding} = \text{kernel_size} // 2 = 2$ is computed automatically to keep sequence length unchanged ('same' padding).
lstm_hidden	128	Number of hidden units in the LSTM cell. With <code>bidirectional=True</code> the LSTM produces $2 * 128 = 256$ features per timestep (forward + backward concatenated).
lstm_layers	1	Number of stacked LSTM layers. With 1 layer the inter-layer dropout argument is forced to 0.0 (PyTorch requirement — dropout between LSTM layers only applies when <code>num_layers > 1</code>).
bidirectional	True	Wraps the LSTM to run in both directions. Forward pass reads $t=0..T$, backward pass reads $t=T..0$. Their hidden states are concatenated, doubling the output dimension to $\text{lstm_hidden} * 2$.
dropout	0.4	Shared dropout rate used throughout: after each Conv block, between LSTM layers (if <code>lstm_layers > 1</code>), and before the final Linear layer. 0.4 is slightly stronger regularisation than the 0.5 in the Keras model.

2. nn.Sequential conv block — overview

A container that chains layers and calls them in order

```
self.conv = nn.Sequential(
    # Block 1
    nn.Conv1d(...), nn.BatchNorm1d(...), nn.ReLU(...),
    nn.MaxPool1d(2), nn.Dropout(dropout),
    # Block 2
    nn.Conv1d(...), nn.BatchNorm1d(...), nn.ReLU(...),
    nn.MaxPool1d(2), nn.Dropout(dropout),
)
```

`nn.Sequential` is a container module that stores an ordered list of child modules. When you call `self.conv(x)` in the forward pass, PyTorch automatically calls each child in the order they were added, passing the output of one as the input to the next. It is equivalent to writing each call out manually in `forward()`, but keeps the constructor tidy and makes the block easy to inspect or replace.

Note that unlike Keras's `model.summary()`, PyTorch does not compute output shapes automatically. You need to trace through the math manually or use a tool like `torchinfo.summary(model, input_size=(B, 2, L))` to see shapes at each layer.

3. Conv Block 1

Conv1d + BatchNorm1d + ReLU + MaxPool1d + Dropout

(B, 2, L) → (B, 64, L) → (B, 64, L) → (B, 64, L/2) → (B, 64, L/2)

```
nn.Conv1d(input_channels, conv_filters, kernel_size, padding=padding),
nn.BatchNorm1d(conv_filters),
nn.ReLU(inplace=True),
nn.MaxPool1d(2),
nn.Dropout(dropout),
```

nn.Conv1d

The PyTorch equivalent of Keras Conv1D, but with a critical difference in tensor layout: PyTorch Conv1d expects input shape (batch, channels, length) — channels come before the time axis. Keras Conv1D expects (batch, length, channels). This is why the `forward()` method calls `x.transpose(1, 2)` before the LSTM.

in_channels	<code>input_channels (2)</code>	Number of input channels. 2 for I and Q.
out_channels	<code>conv_filters (64)</code>	Number of learned filters (output channels).
kernel_size	5	Each filter spans 5 consecutive timesteps. Larger than the Keras model's 3 — sees slightly more context per convolution.
padding	<code>kernel_size // 2 = 2</code>	Computed in <code>__init__</code> as <code>kernel_size // 2</code> . For odd kernel sizes this gives exact 'same' padding: output length equals input length.

nn.BatchNorm1d

Normalises the activations across the batch dimension for each channel independently. For a tensor of shape (B, C, L) , it normalises along the B and L axes for each of the C channels. Placed after Conv and before ReLU here (pre-activation style — slightly different from the post-activation placement in the Keras CNN1D model, but both are valid conventions).

nn.ReLU(inplace=True)

Standard ReLU activation. `inplace=True` modifies the tensor in memory rather than allocating a new one, saving a small amount of GPU memory. This is safe here because no other part of the graph needs the pre-ReLU values. Functionally identical to `inplace=False`.

nn.MaxPool1d(2)

Halves the sequence length by taking the maximum over non-overlapping windows of 2. After this layer: $(B, 64, L/2)$. Default stride equals `kernel_size`, so the windows do not overlap.

nn.Dropout(dropout)

Placed after MaxPool rather than after Dense (as in the Keras model). In PyTorch, `nn.Dropout` applied to a 3-D tensor (B, C, L) zeroes entire elements randomly. This means some (channel, timestep) pairs are silenced, providing regularisation throughout the feature map. Like all PyTorch modules, it is automatically disabled during `model.eval()`.

4. Conv Block 2

Deeper filters — `conv_filters * 2 = 128` output channels

$(B, 64, L/2)$ $\rightarrow$ $(B, 128, L/2)$ $\rightarrow$ $(B, 128, L/4)$

```
nn.Conv1d(conv_filters, conv_filters * 2, kernel_size, padding=padding),
nn.BatchNorm1d(conv_filters * 2),
nn.ReLU(inplace=True),
nn.MaxPool1d(2),
nn.Dropout(dropout),
```

Identical structure to Block 1 with two differences. First, `in_channels` is now `conv_filters` (64) — the output of Block 1 feeds in. Second, `out_channels` is `conv_filters * 2` (128). Doubling the filter count compensates for the halved spatial resolution: the model sacrifices temporal precision for representational depth, which is the standard CNN design trade-off.

After Block 2, the tensor is $(B, 128, L/4)$. For a standard RadioML 2016 input of $L=128$ timesteps, this becomes $(B, 128, 32)$ — 32 temporal positions, each described by 128 feature channels. This compressed representation is what the LSTM will read as a sequence.

5. nn.LSTM

Sequential modelling — reads the CNN features as a time series

(B, 32, 128)

->

(B, 32, 256)
[bidirectional]

```
self.lstm = nn.LSTM(
    input_size=conv_filters * 2,          # 128
    hidden_size=lstm_hidden,              # 128
    num_layers=lstm_layers,               # 1
    batch_first=True,
    bidirectional=bidirectional,          # True
    dropout=dropout if lstm_layers > 1 else 0.0,
)
```

What the LSTM does. A Long Short-Term Memory cell processes the sequence one timestep at a time, maintaining a hidden state h_t and a cell state c_t that carry information across steps. Three gating mechanisms — input gate, forget gate, output gate — control how much of the previous state is retained and how much new information is written. This allows the LSTM to capture dependencies that span many timesteps, which a pure CNN cannot do without very large kernels.

In this model the LSTM reads the 32-step feature sequence produced by the CNN. Each 'timestep' is a 128-dimensional vector of CNN features, not raw I/Q samples. The LSTM therefore reasons about *how the CNN features evolve* along the signal — a higher-level form of sequence modelling.

LSTM parameters

input_size	<code>conv_filters * 2 = 128</code>	Dimensionality of each input vector at each timestep — must match the channel dimension of the transposed conv output.
hidden_size	<code>lstm_hidden = 128</code>	Number of features in the hidden state h_t . With <code>bidirectional=True</code> , the final output concatenates forward and backward hidden states: effective size is 256.
num_layers	<code>lstm_layers = 1</code>	Number of stacked LSTM layers. With 1, the output goes directly to the classifier. With 2+, the hidden state of one layer feeds the next, and inter-layer dropout applies.
batch_first	<code>True</code>	When True, expects input shape (B, T, features) instead of the default (T, B, features). This is why <code>transpose(1,2)</code> in <code>forward()</code> swaps from (B, C, L) to (B, L, C) before passing to the LSTM.

bidirectional	True	Runs two LSTMs in parallel — one reading left to right, one right to left. Their outputs are concatenated at each timestep, giving shape (B, T, 2*hidden_size). This lets the model see future context when deciding what a given timestep means.
dropout	0.0 (lstm_layers=1)	Inter-layer dropout, applied between stacked LSTM layers only. PyTorch requires this to be 0.0 when num_layers=1 (there are no inter-layer connections to drop). The conditional dropout if lstm_layers > 1 else 0.0 handles this automatically.

6. forward() — the data flow step by step

Every line of the forward pass explained

```
def forward(self, x: torch.Tensor) -> torch.Tensor:
    x = self.conv(x)          # (B, 2, L) -> (B, 128, L/4)
    x = x.transpose(1, 2)     # (B, 128, L/4) -> (B, L/4, 128)
    output, _ = self.lstm(x)  # (B, L/4, 128) -> (B, L/4, 256)
    x = output.mean(dim=1)    # (B, L/4, 256) -> (B, 256)
    return self.classifier(x) # (B, 256) -> (B, 24)
```

Step 1: `x = self.conv(x)`

Runs the entire nn.Sequential conv block. Input shape (B, 2, L) — batch, 2 channels (I/Q), L timesteps. Output shape (B, 128, L/4) after two rounds of convolution and max-pooling. For L=128 this gives (B, 128, 32).

Step 2: `x = x.transpose(1, 2)`

Swaps the channel and time axes. PyTorch Conv1d uses (B, C, L) layout, but nn.LSTM with batch_first=True expects (B, L, C). After transpose: (B, 32, 128). Each of the 32 positions is now a 128-dimensional feature vector — one LSTM input per CNN timestep.

Step 3: `output, _ = self.lstm(x)`

Runs the bidirectional LSTM over the 32-step sequence. Returns a tuple (all_hidden_states, (h_n, c_n)). output shape: (B, 32, 256) — 256 because bidirectional concatenates 128 forward + 128 backward hidden states at each step. The final states tuple (h_n, c_n) is discarded with _ because we want all timestep outputs, not just the last.

Step 4: `x = output.mean(dim=1)`

Averages the LSTM output across all 32 timesteps. Shape: (B, 256). This is temporal mean pooling — equivalent to GlobalAveragePooling1D in Keras. It collapses the sequence into a single fixed-size vector, regardless of input length L. An alternative is to take only the final hidden state, but mean pooling tends to be more robust because it incorporates information from all timesteps.

Step 5: `return self.classifier(x)`

Passes the 256-dimensional vector through LayerNorm, Dropout, and Linear to produce the final (B, 24) logit vector. No softmax here — PyTorch's `nn.CrossEntropyLoss` applies log-softmax internally, so the raw logits are the correct output for training.

7. Classifier head

LayerNorm + Dropout + Linear — maps LSTM features to class logits

(B, 256) -> (B, 256) -> (B, 256) -> (B, 24)

```
directions = 2 if bidirectional else 1
self.classifier = nn.Sequential(
    nn.LayerNorm(lstm_hidden * directions), # 256
    nn.Dropout(dropout),                  # 0.4
    nn.Linear(lstm_hidden * directions, num_classes), # 256 -> 24
)
```

nn.LayerNorm(256)

Normalises across the feature dimension (the 256 values) for each example independently. Unlike BatchNorm, LayerNorm does not depend on other examples in the batch, making it well-suited after recurrent layers where batch statistics can be noisy. It is also unaffected by batch size, which matters during inference when you may process one signal at a time.

The `lstm_hidden * directions = 128 * 2 = 256` calculation matches the output dimension of the bidirectional LSTM. If you set `bidirectional=False`, `directions` becomes 1 and all subsequent dimensions halve to 128 automatically.

nn.Dropout(0.4)

Standard dropout applied to the normalised 256-d vector before the final linear projection. Randomly zeroes 40% of the values each training step. As with all PyTorch modules, it respects `model.train()` and `model.eval()` — you must call `model.eval()` before inference to disable it.

nn.Linear(256, num_classes)

A fully-connected projection from 256 to `num_classes` (24). Equivalent to Keras `Dense(24)` but without a softmax activation. The output is raw logits. Use with `nn.CrossEntropyLoss()` during training, which applies log-softmax internally. For inference, wrap with `torch.softmax(logits, dim=-1)` or `torch.argmax(logits, dim=-1)` for class predictions.

8. Key differences: PyTorch vs Keras

What changes when moving from the Keras CNN1D to this PyTorch model

Topic	Keras CNN1D	PyTorch CNN-LSTM
Tensor layout	(B, L, C) — length then channels	(B, C, L) — channels then length
Activation	activation= argument in layer	Separate nn.ReLU() module
Pooling norm order	Conv -> ReLU -> Pool -> BN	Conv -> BN -> ReLU -> Pool -> Dropout
Softmax	softmax in final Dense layer	No softmax — logits only; use CrossEntropyLoss
Dropout in Conv	After Dense head only	After every Conv block + classifier
Normalisation head	No normalisation before output	LayerNorm before Linear
LSTM output	return_sequences=False (last state)	output.mean(dim=1) — temporal mean pool
Training mode	Automatic in model.fit()	Must call model.train() / model.eval() manually
Loss function	categorical_crossentropy	nn.CrossEntropyLoss (integer labels, not one-hot)

9. Full model code

cnn_lstm.py — reproduced verbatim

```

class CNNLSTM(nn.Module):
    def __init__(
        self,
        num_classes: int = 24,
        input_channels: int = 2,
        conv_filters: int = 64,
        kernel_size: int = 5,
        lstm_hidden: int = 128,
        lstm_layers: int = 1,
        bidirectional: bool = True,
        dropout: float = 0.4,
    ) -> None:
        super().__init__()
        padding = kernel_size // 2

        self.conv = nn.Sequential(
            nn.Conv1d(input_channels, conv_filters, kernel_size, padding=padding),
            nn.BatchNorm1d(conv_filters),
            nn.ReLU(inplace=True),
            nn.MaxPool1d(2),
            nn.Dropout(dropout),
            nn.Conv1d(conv_filters, conv_filters * 2, kernel_size, padding=padding),
            nn.BatchNorm1d(conv_filters * 2),
            nn.ReLU(inplace=True),
            nn.MaxPool1d(2),
            nn.Dropout(dropout),
        )

        self.lstm = nn.LSTM(
            input_size=conv_filters * 2,
            hidden_size=lstm_hidden,
            num_layers=lstm_layers,
            batch_first=True,
            bidirectional=bidirectional,
            dropout=dropout if lstm_layers > 1 else 0.0,
        )

        directions = 2 if bidirectional else 1
        self.classifier = nn.Sequential(
            nn.LayerNorm(lstm_hidden * directions),
            nn.Dropout(dropout),
            nn.Linear(lstm_hidden * directions, num_classes),
        )

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        x = self.conv(x)
        x = x.transpose(1, 2)
        output, _ = self.lstm(x)
        x = output.mean(dim=1)
        return self.classifier(x)

```

